9주차 : 자연어처리와 컴퓨터비전 심층학습 10장

구분	CNN	Transformer
핵심 연산	Convolution	Attention
특징 추출	주변 영역 중심	데이터 요소 간 관계 중심
먼 위치의 정보	여러 Layer를 거쳐 전달 가능	Attention으로 직접 연결 가능
대표 모델	ResNet, U-Net	BERT, GPT, ViT

- 트랜스포머 (Transformer) : 자연어 처리(NLP)를 위해 2017년에 제안된 모델
  - Self-Attention : 단어 하나만 독립적으로 보는 것이 아니라 다른 단어들을 함께 참고하여 의미를 학습함 → 이를 통해 문장 속 단어들이 서로 얼마나 관련 있는지 계산함
  - Attention Weight : 각 단어가 다른 단어들을 얼마나 참고해야 하는지 결정
  - 트랜스포머의 기본 구조
    - Embedding: 단어나 Patch를 숫자 벡터로 변환
    - Positional Encoding: 단어나 Patch의 위치 정보 추가
    - Multi-Head Self-Attention: 여러 관점에서 데이터 사이의 관계 학습
    - Feed Forward Network (FFN): Attention으로 얻은 특징을 추가적으로 변환
    - Residual Connection & Layer Normalization: 학습을 안정적으로 진행

- ViT (Vision Transformer) : 이미지 인식을 위한 딥러닝 모델
  → 이미지를 자연어 처리 방식처럼 분류해 보려는 시도에 의해 탄생

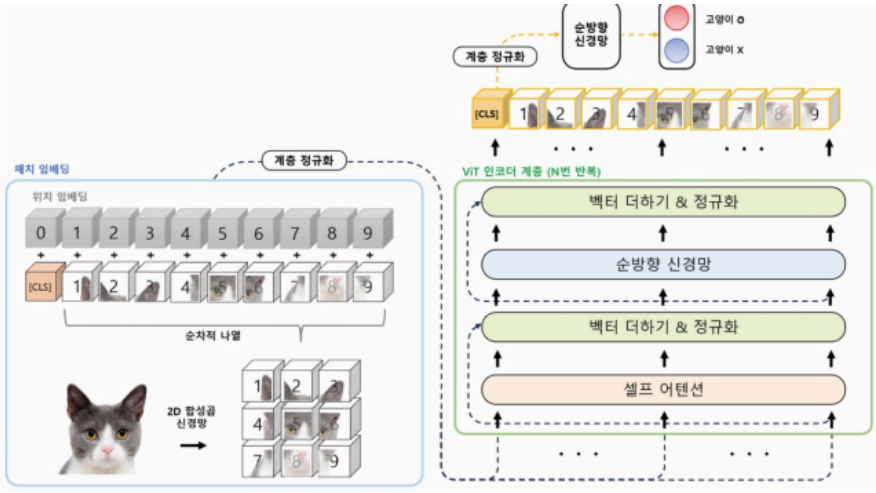


그림 10.4 ViT 모델 구조

- 셀프 어텐션 적용 : 전체 이미지를 한번에 처리하는 방식으로 구현
  ↔ 합성곱 모델 : 이미지를 분류하기 위해 지역 특징을 추출 (CNN 모델)
- 이미지 패치가 왼쪽에서 오른쪽, 위에서 아래의 방향으로 순차적으로 입력되어 2차원 구조의 이미지 특성을 온전히 반영한다고 할 수 없음
  → 스윈 트랜스포머(Swin Transformer), CvT(Convolution Vision Transformer) 모델이 제안됨
  - 스윈 트랜스포머 : Local Window를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성해 이미지의 특징을 학습시킴
  - CvT : 기존 합성곱 연산 과정을 ViT에 적용 → 저수준 특징과 저수준 특징을 계층적으로 반영 가능

- ViT 모델의 계층 구조 : 패치 임베딩과 인코더 계층으로 구성됨
  - 패치 임베딩 : 입력 이미지를 작은 패치로 분할하는 과정

  a. 작은 패치로 분할하기 위해서는 이미지 크기를 맞추는 전처리가 수행되어야 함

  b. 이미지 크기를 일정한 크기로 변경한 후 전체 이미지를 패치 크기로 분할해 시퀀셜 배열을 만듦
  → 이때 합성곱 신경망의 계층을 활용함

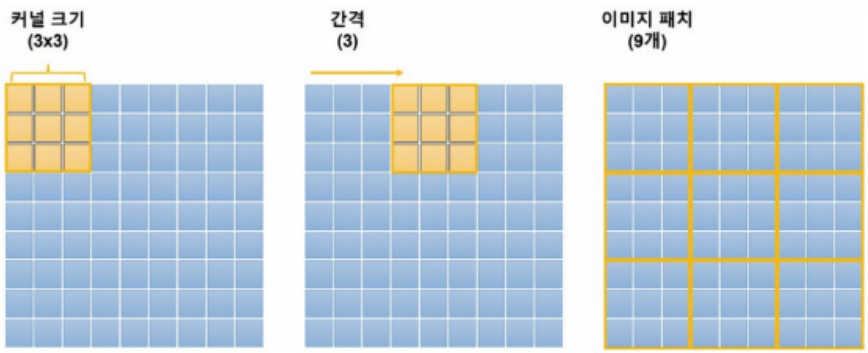


그림 10.5 이미지 패치 생성 과정

수식 10.1 패치 계산 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9-3}{3} + 1 = 3$$

(가로 패치 3개, 세로 패치 3개 → 총 9개의 패치 생성)

- 인코더 계층 : 각 패치를 벡터로 변환한 후 인코더 계층에 입력으로 전달해 다양한 쿼리, 키, 값 임베딩의 관계를 학습함

- CNN 모델과 ViT 모델의 비교
  - CNN 임베딩 : 이미지 패치 중 일부만 선택하여 학습 → 이를 통해 이미지 전체의 특징을 추출
  - ViT 임베딩 : 이미지를 작은 패치들로 나눠 각 패치 간의 상관관계 학습

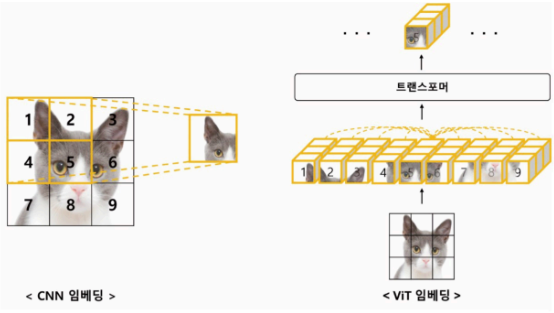


그림 10.3 합성곱 신경망과 ViT 계층의 차이

비교 기준	CNN	ViT
핵심 연산	Convolution	Self-Attention
입력 처리	이미지에 Filter 적용	이미지를 Patch로 분할
특징 추출	Local Feature 중심	Global Relationship 학습
위치 정보	이미지의 공간 구조를 활용	Position Embedding 활용
Inductive Bias	상대적으로 강함	상대적으로 약함
데이터 요구량	적은 데이터에서도 비교적 유리	일반적으로 대규모 사전학습이 중요
대표 모델	ResNet, VGG, U-Net	ViT, DeiT

- BERT 모델과 ViT 모델의 비교
  - BERT 모델 : 문장보다 작은 단위의 토큰들이 순차적으로 입력됨
  - ViT모델 : 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨

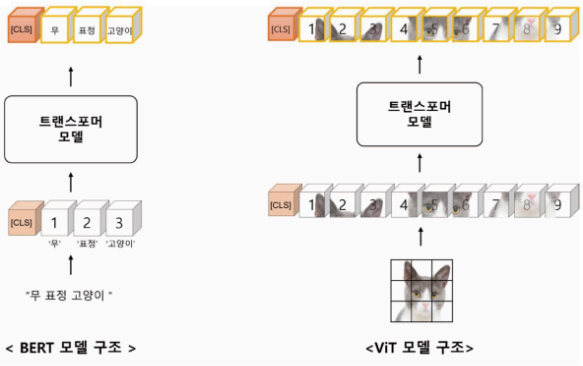
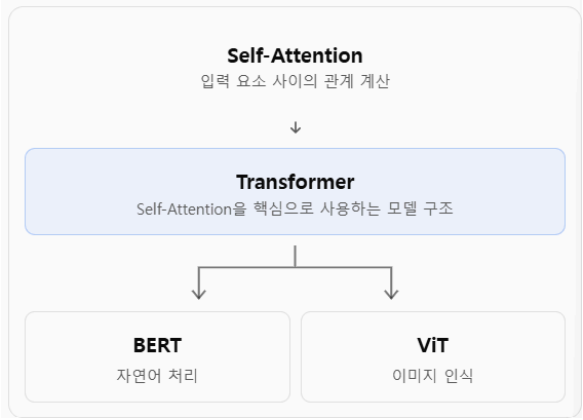


그림 10.2 BERT의 ViT 모델 구조 차이



- ViT의 귀납적 편향 (Inductive Bias)
  - 귀납적 편향 : 모델이 데이터를 학습할 때 미리 가지고 있는 기본적인 가정
    ex) [CNN] 가까운 픽셀끼리는 관련성이 높을 것이다
    ex) [CNN] 이미지의 어느 위치에서든 동일한 특징(예 : 모서리)을 찾는 것이 유용할 것이다.

표 10.1 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

  - ViT는 입력 데이터의 다양한 쿼리, 키, 값의 임베딩 형태로 일반화된 관계를 학습하기 때문에 귀납적 편향이 거의 없고, 대용량 데이터에 대해서 이미지 특징들의 관계를 잘 학습하는 모델로 알려져 있음

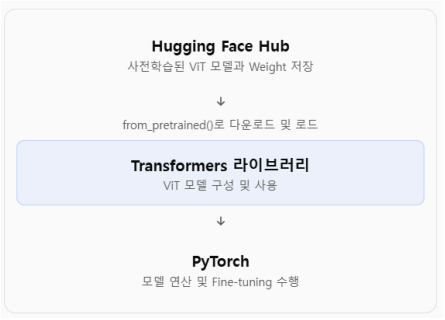
구성	의미	직관적인 역할
Query (Q)	내가 찾는 정보	무엇을 참고할까?
Key (K)	내가 가진 정보의 특징	나와 얼마나 관련 있는가?
Value (V)	실제 전달할 정보	어떤 정보를 가져올까?

  - ViT는 CNN에 비해 상대적으로 귀납적 편향이 약하다 : CNN처럼 가까운 픽셀끼리 중요하다는 가정을 강하게 적용하지 않음

→ 대규모 데이터에서 다양한 관계를 학습하는 데 강점이 있지만, 충분한 학습 데이터나 사전 학습이 중요할 수 있음

- Fashion MNIST 실습
  - Hugging Face Hub의 역할 : 다른 사람들이 학습 시켜 놓은 AI 모델과 데이터셋을 공유하고, 가져다 사용할 수 있는 온라인 플랫폼 (= AI 모델을 위한 GitHub)

기능	설명	예시
Models	사전학습된 AI 모델 공유	BERT, ViT, Llama
Datasets	AI 학습용 데이터셋 공유	이미지, 텍스트, 음성 데이터
Spaces	AI 모델을 체험할 수 있는 웹 데모 공유	이미지 분류, 챗봇



- 스윈 트랜스포머 : 대규모 비전 인식 모델
  - 시프트 윈도(Shifted Window) 기술 이용 : 입력 이미지를 일정한 크기의 패치로 분할하고 이 패치들을 쌓아 윈도를 구성함 → 구성된 윈도 영역만 셀프 어텐션 계산
  - 로컬 윈도(Local Window) 기술 이용 : 물체의 크기나 해상도를 계층적으로 학습
    → 로컬 윈도 : 스윈 트랜스포머에서 입력 이미지를 처리하기 위해 사용되는 고정 크기의 작은 윈도

⇒ 입력 패치의 로컬 윈도를 통해 계층적으로 접근하고 시프트 윈도로 로컬 윈도를 이동해 가면서 인접 패치 간 정보를 계산한다.

  - ViT와 스윈 트랜스포머의 차이

구분	ViT	Swin Transformer
핵심 연산	Global Self-Attention	Window-based Self-Attention
Attention 범위	전체 이미지의 Patch	각 Window 내부의 Patch
Shifted Window	기본 구조에는 없음	핵심 기술
특징	전체 Patch 사이의 관계 학습	국소 영역 학습 + Window 사이의 정보 교환

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

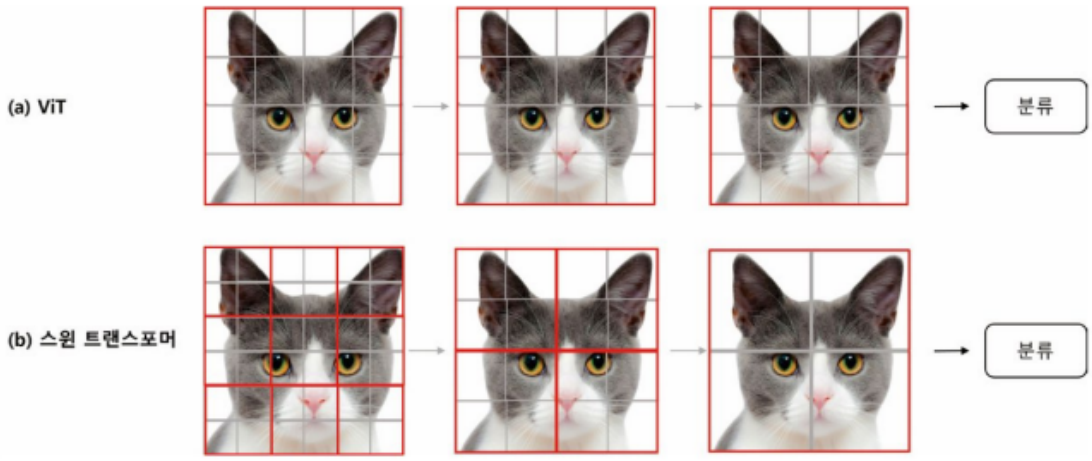


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

- 회색 격자 : 패치, 빨간색 격자 : 로컬 윈도

- 스윈 트랜스포머 모델 구조 : 아래 3가지 연산이 반복적으로 수행
  1. 선형 임베딩 (Linear Embedding)
  2. 스윈 트랜스포머 블록 (Swin Transformer Block)
  3. 패치 병합 (Patch Merging)

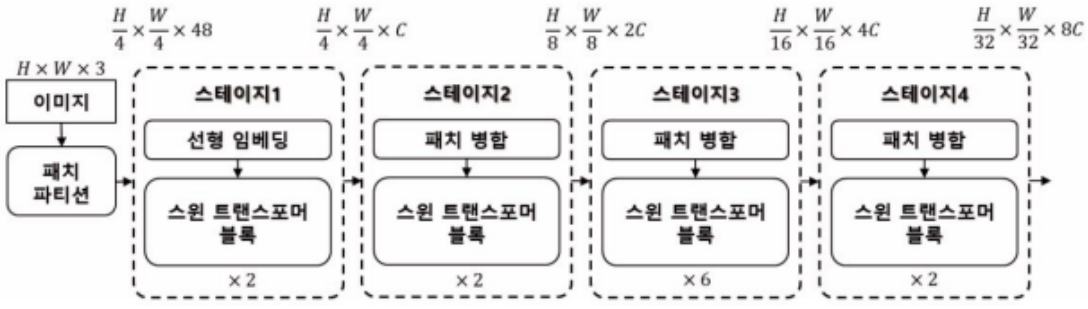


그림 10.7 스윈 트랜스포머 구조

- 패치 파티션 : 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식
  → 분할된 패치는 트랜스포머 계층의 입력으로 사용됨

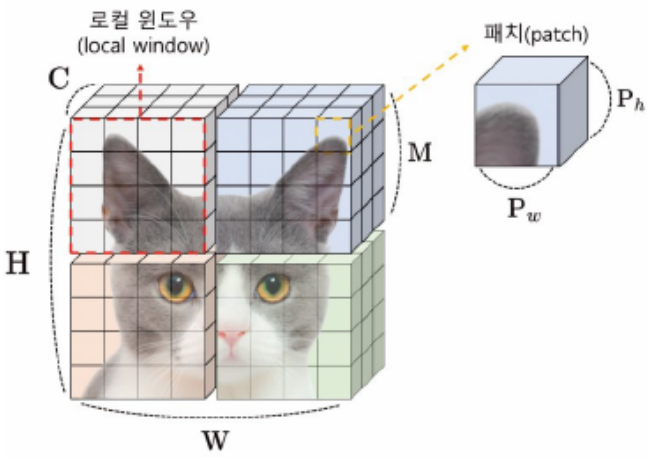


그림 10.8 패치 파티션 구조

- 패치 병합 : 인접한 패치들의 정보를 저차원으로 축소하는 과정
- 스윈 트랜스포머 블록 : 패치 파티션 이후 네 개의 스테이지 안에서 반복 적용
  → 선형 임베딩 또는 패치 병합 이후에 수행됨

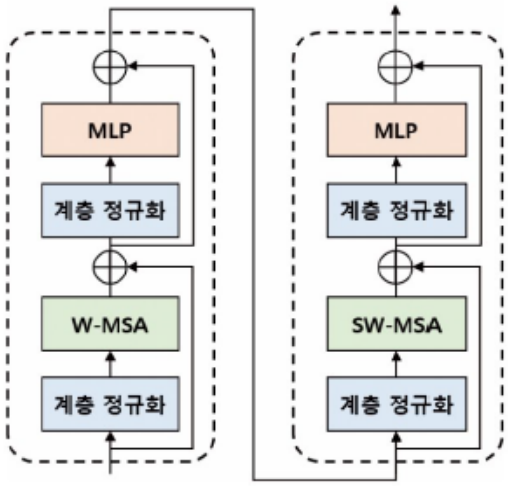


그림 10.9 스윈 트랜스포머 블록 구조

- W-MSA : 로컬 윈도를 사용하는 멀티 헤드 셀프 어텐션
  → 입력 특징 맵을 중첩되지 않게 나눈 다음 각 윈도에서 독립적으로 셀프 어텐션 수행
  → 효율적인 배치 계산을 통해 연산량은 감소했지만, 윈도 내부 이미지 패치 영역에만 셀프 어텐션을 수행한다는 단점이 있음

- SW-MSA : 윈도 사이의 연결성을 구축해 윈도 간의 관계성 정보를 추출함
  → 순환 시프트, 텐션 마스크, 역순환 시프트 사용

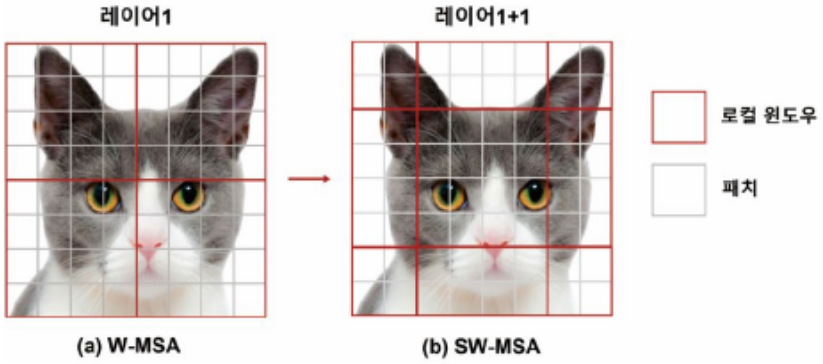


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

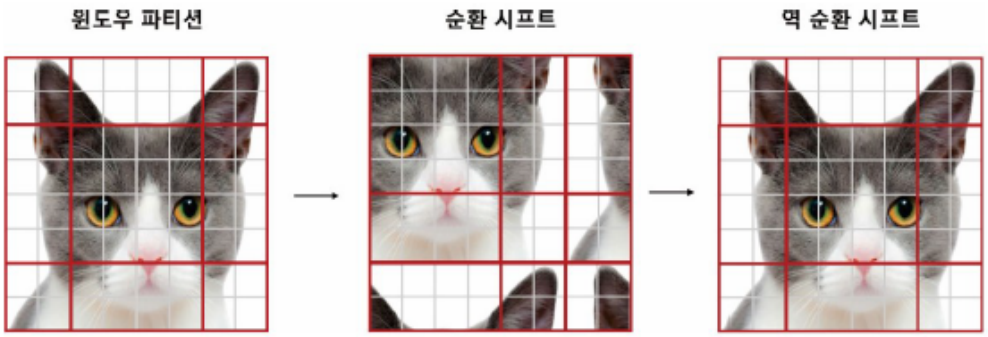


그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

- 상대적 위치 편향 (Relative Position Bias) : 로컬 윈도 안에 있는 패치 간의 상대적인 거리 임베딩
  → 기존 ViT와 다른 점 : 스윈 트랜스포머에서 사용한 셀프 어텐션은 상대적 위치 편향을 고려함



그림 10.13 상대적 위치 편향 계산 과정

- CvT(Convolutional Vision Transformer) : 합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델 → 이미지의 지역 특징과 전역 특징을 모두 활용해 비교적 적은 수의 매개변수로 높은 성능을 보임-

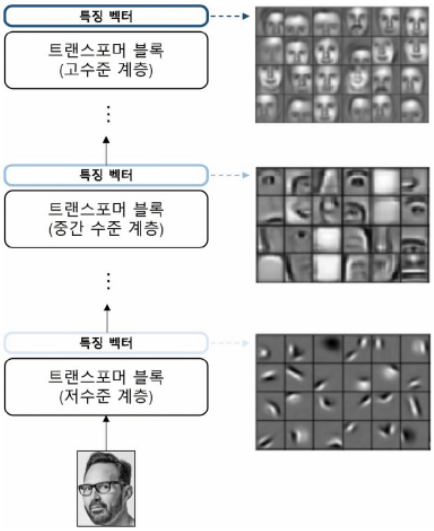


그림 10.14 CvT 모델의 계층적 특징 구조 예시

표 10.4 ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

- 합성곱 토큰 임베딩 : 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩
  → ViT에 계층적인 합성곱 신경망의 성질을 추가하기 위해 사용

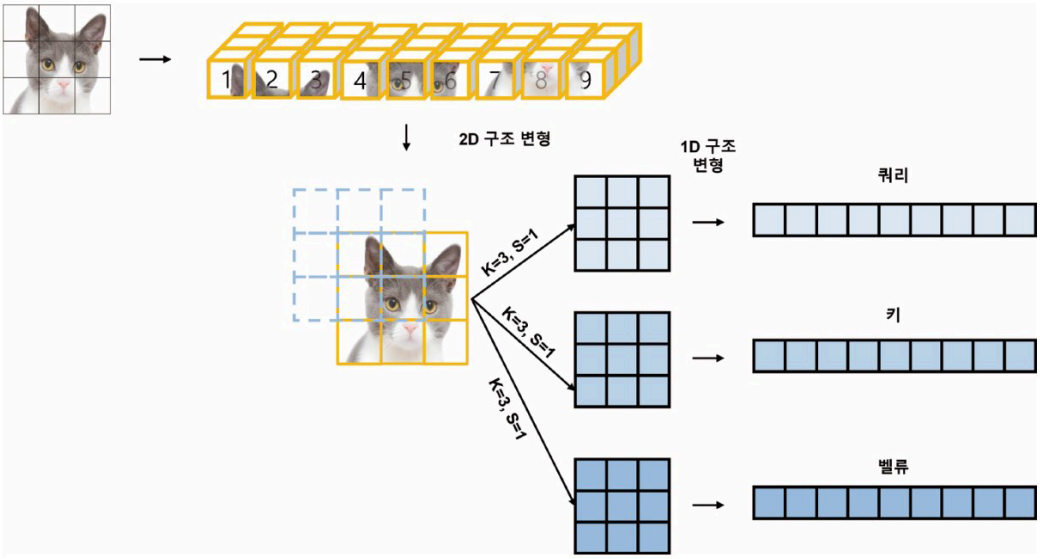


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

- 어텐션에 대한 합성곱 임베딩 : 2D로 재구성된 토큰 맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산

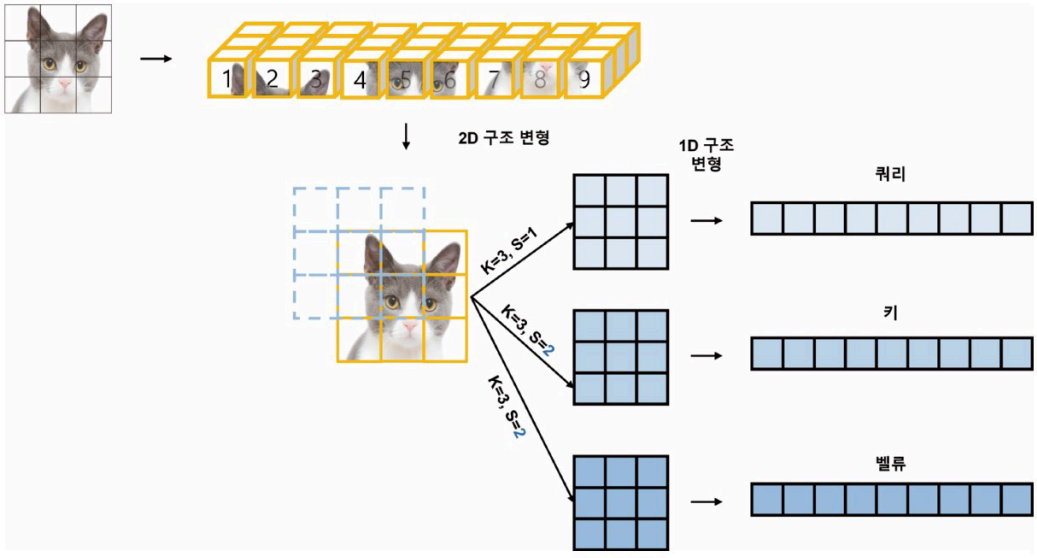


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

▼ Week9 코드필사

https://colab.research.google.com/drive/1FX42oRRmXWW4W495vnW__tzVe_4MLfcH?usp=drive_link